HyperionDev

API Integration

Practical Skills Module 5: Topic 1

Visit our website

Practical Skills Module 5: Topic 1

Topic Code	PM-05-PS01
Topic	Start using API with Python

Scope of Practical Skill

Given an applicable instruction and access to a learning platform, the learner must be able to:

- PA0101 Get an API key
- PA0102 Test API endpoints
- PA0103 Retrieve information – GET
- PA0104 Add new data – POST
- PA0105 Change existing information – PUT
- PA0106 Delete existing information – DELETE
- PA0107 View the status codes of the response
- PA0108 Generate a Python snippet

Applied Knowledge

- AK0101 Concept, definition and functions of REST API

Internal Assessment Criteria

- IAC0101 Expected results with Python REST API are achieved

Copyright © 2025 HyperionDev. All rights reserved.

Resources:

Knowledge	Information from Python Programmer Curriculum
National Curriculum Framework	900221-000-00-PM-05



Practical task

Follow the facilitator's instructions to complete the following activities:

Scenario:
A **weather monitoring application** interacts with a weather API to fetch real-time weather data. In a real-world application, user preferences (such as preferred city, temperature units, contact method etc.) would typically be stored and managed on your own backend server, not through the weather API. Since we're not building a backend in this task, you will simulate user preference actions using a **mock API** such as JSONPlaceholder, which mimics real API behavior.

In this task, you will:

- Use the weather API (e.g., OpenWeatherMap) for retrieving weather data.
- Simulate adding, updating, and deleting user preferences using a mock API.
- Handle API response status codes appropriately in your application.
- Generate Python code snippets to automate these tasks.

Create Python file called **weather_api.py** and complete the following:

Step 1: Get an API key

- Sign up for a weather API service (e.g., **OpenWeatherMap**) and generate an API key to authenticate requests.

Step 2: Test API endpoints

- Verify that the weather API is accessible and your API key is working by making a simple GET request with a test city (for example: London).
- You can use parameters like **"q"** (representing the city name) and **"appid"** (representing your API key) to retrieve location information from the API.
- Explore other parameters you can use to access more information from this **resource**.

Copyright © 2025 HyperionDev. All rights reserved.

Step 3: Retrieve information – GET

- Fetch current weather information for any city (for example: New York).

Step 4: Add new data – POST

- Simulate saving a new user's weather notification preferences using the mock API.

Step 5: Change existing information – PUT

- Simulate updating the notification preferences for a user using the mock API.

Step 6: Delete existing information – DELETE

- Simulate removing a user's notification preferences using the mock API.

Step 7: View the status codes of the response

- Check and handle response status codes after each API interaction to confirm success or manage errors.

Step 8: Generate a Python snippet

- Generate reusable Python code to automate interactions with the weather API.



Take note

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.

Copyright © 2025 HyperionDev. All rights reserved.